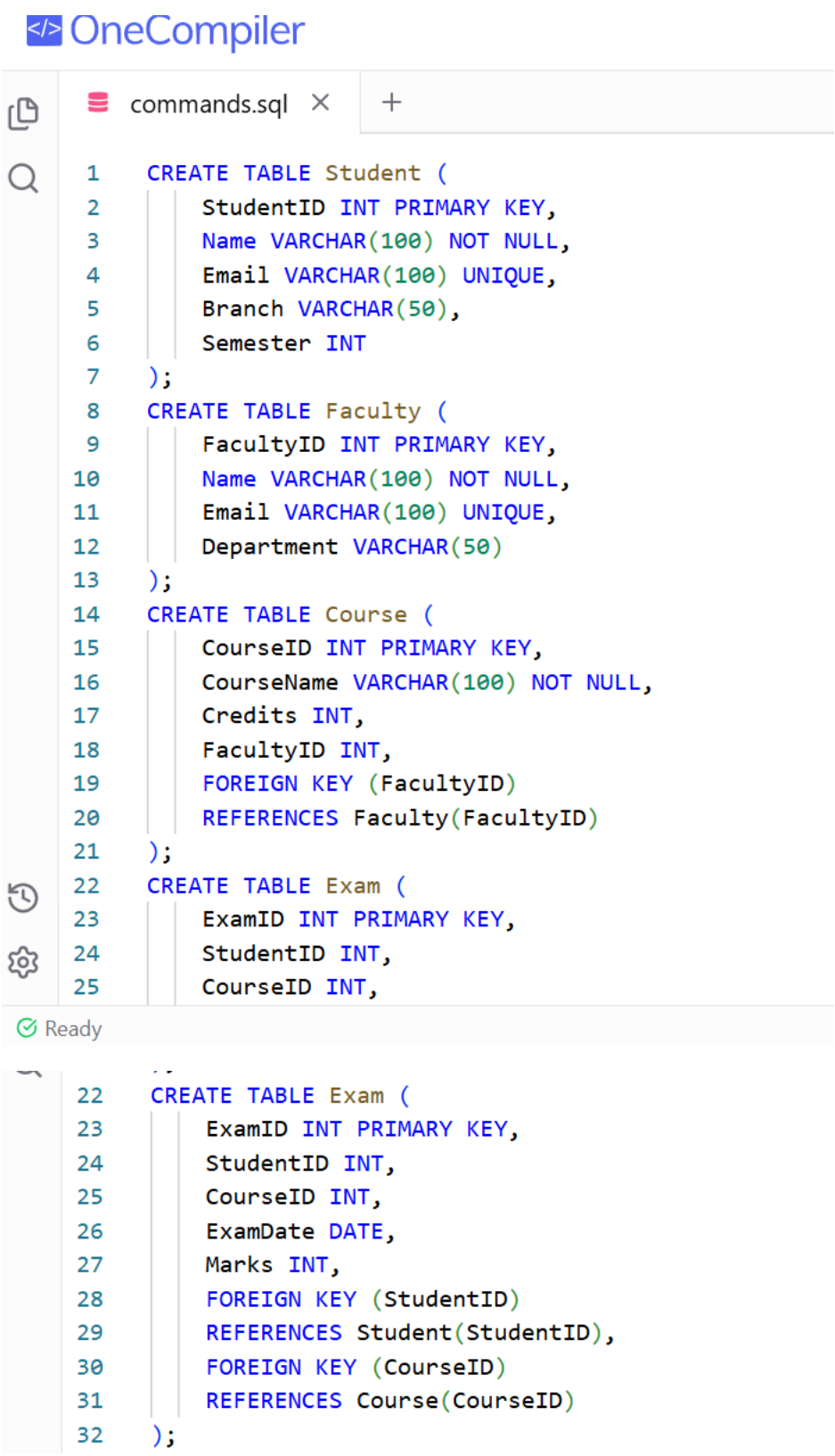
Lab 2:Building and Quering a Database

1. DDL Statements to Query the Table-

Write DDL statements to create the four tables above. Each table must have a Primary Key. Course.FacultyID must be a Foreign Key referencing Faculty; Exam.StudentID and Exam.CourseID must be Foreign Keys referencing Student and Course respectively. Apply NOT NULL on Name fields and UNIQUE on an appropriate alternate column. Run the table creation and confirm with DESCRIBE/\d output for all four tables before proceeding.

STEP1 Create the four tables:-



Execute the SQL commands

Output:

```
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
```

STEP2 Check all four tables:-

```
);
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;
```

```
 course
 exam
 faculty
 student
(4 rows)
```

STEP3 Describe all four tables:-

```
33  SELECT column_name, data_type, is_nullable
34  FROM information_schema.columns
35  WHERE table_name = 'student'
36  ORDER BY ordinal_position;
```

```
 column_name |     data_type     | is_nullable
-------------+-------------------+-------------
 studentid   | integer           | NO
 name        | character varying | NO
 email       | character varying | YES
 branch      | character varying | YES
 semester    | integer           | YES
(5 rows)
```

```sql
SELECT
    tc.table_name,
    tc.constraint_name,
    tc.constraint_type
FROM information_schema.table_constraints tc
WHERE tc.table_schema = 'public'
ORDER BY tc.table_name;
```

```
 table_name |      constraint_name       | constraint_type
------------+----------------------------+-----------------
 course     | course_courseid_not_null   | CHECK
 course     | course_facultyid_fkey      | FOREIGN KEY
 course     | course_pkey                | PRIMARY KEY
 course     | course_coursename_not_null | CHECK
 exam       | exam_courseid_fkey         | FOREIGN KEY
 exam       | exam_examid_not_null       | CHECK
 exam       | exam_pkey                  | PRIMARY KEY
 exam       | exam_studentid_fkey        | FOREIGN KEY
 faculty    | faculty_facultyid_not_null | CHECK
 faculty    | faculty_name_not_null      | CHECK
 faculty    | faculty_pkey               | PRIMARY KEY
 faculty    | faculty_email_key          | UNIQUE
 student    | student_studentid_not_null | CHECK
 student    | student_email_key          | UNIQUE
 student    | student_pkey               | PRIMARY KEY
 student    | student_name_not_null      | CHECK
(16 rows)
```

2. Inserting Data in to Tables, Update Data-

(Requires Q1 tables to exist) Insert at least 8 Students, 4 Faculty, 5 Courses and 12 Exam records, ensuring enough variety to support later GROUP BY and JOIN queries (e.g., multiple students per course, at least one student with no exam record yet). Then: (a) UPDATE one student's Branch, (b) DELETE one Exam record, (c) attempt to INSERT an

Exam record with a non-existent StudentID and show/explain the resulting foreign key error.

STEP-1 Insert data into Student table:-

```
INSERT INTO Student (StudentID, Name, Email, Branch, Semester)
VALUES
(1, 'Rahul Kumar', 'rahul@gmail.com', 'MCA', 1),
(2, 'Priya Sharma', 'priya@gmail.com', 'MCA', 1),
(3, 'Aman Verma', 'aman@gmail.com', 'BCA', 2),
(4, 'Neha Singh', 'neha@gmail.com', 'MCA', 2),
(5, 'Rohit Gupta', 'rohit@gmail.com', 'BCA', 3),
(6, 'Anjali Mehta', 'anjali@gmail.com', 'MCA', 1),
(7, 'Karan Malhotra', 'karan@gmail.com', 'BCA', 2),
(8, 'Sneha Jain', 'sneha@gmail.com', 'MCA', 1);
```

```
INSERT 0 8
```

```
INSERT INTO Faculty (FacultyID, Name, Email, Department)
VALUES
(101, 'Dr. Amit Sharma', 'amit@gmail.com', 'Computer Science'),
(102, 'Dr. Pooja Verma', 'pooja@gmail.com', 'Mathematics'),
(103, 'Dr. Rajesh Kumar', 'rajesh@gmail.com', 'Information Technology'),
(104, 'Dr. Neelam Singh', 'neelam@gmail.com', 'Artificial Intelligence');
```

```
INSERT 0 4
```

```
(104, 'Dr. Neelam Singh', 'neelam@gmail.com', 'Artificial Intelligence');

INSERT INTO Course (CourseID, CourseName, Credits, FacultyID)
VALUES
(201, 'Data Structures', 4, 101),
(202, 'Database Management System', 4, 103),
(203, 'Python Programming', 3, 101),
(204, 'Mathematics', 3, 102),
(205, 'Artificial Intelligence', 4, 104);
```

```
INSERT 0 5
```

```
INSERT INTO Exam (ExamID, StudentID, CourseID, ExamDate, Marks)
VALUES
(301, 1, 201, '2026-10-10', 85),
(302, 1, 202, '2026-10-11', 90),
(303, 2, 201, '2026-10-10', 78),
(304, 2, 203, '2026-10-12', 88),
(305, 3, 202, '2026-10-11', 65),
(306, 3, 204, '2026-10-13', 72),
(307, 4, 201, '2026-10-10', 92),
(308, 4, 205, '2026-10-14', 89),
(309, 5, 203, '2026-10-12', 75),
(310, 6, 204, '2026-10-13', 95),
(311, 6, 205, '2026-10-14', 81),
(312, 7, 202, '2026-10-11', 68);
```

```
INSERT 0 12
```

STEP 2 UPDATE one student's Branch:-

```
UPDATE Student
SET Branch = 'Computer Science'
WHERE StudentID = 1;
```

```
UPDATE 1
```

```
SELECT * FROM Student
WHERE StudentID = 1;
```

```
 studentid |    name     |       email       |      branch      | semester
-----------+-------------+-------------------+------------------+----------
         1 | Rahul Kumar | rahul@gmail.com | Computer Science |        1
(1 row)
```

STEP 3 DELETE one Exam record:-

```
DELETE FROM Exam
WHERE ExamID = 312;
```

```
---------

(1 row)

DELETE 1
```

STEP4 Test Foreign Key constraint (Invalid INSERT):-

```sql
INSERT INTO Exam
(ExamID, StudentID, CourseID, ExamDate, Marks)
VALUES
(313, 999, 201, '2026-10-15', 80);
```

```
psql:commands.sql:102: ERROR:  insert or update on table "exam" violates forei
DETAIL:  Key (studentid)=(999) is not present in table "student".
```

Debug this error

STEP5 Verify all inserted records:-

```sql
SELECT * FROM Student;
```

```
 studentid |      name      |      email       |      branch      | semester
-----------+----------------+------------------+------------------+----------
         2 | Priya Sharma   | priya@gmail.com  | MCA              |        1
         3 | Aman Verma     | aman@gmail.com   | BCA              |        2
         4 | Neha Singh     | neha@gmail.com   | MCA              |        2
         5 | Rohit Gupta    | rohit@gmail.com  | BCA              |        3
         6 | Anjali Mehta   | anjali@gmail.com | MCA              |        1
         7 | Karan Malhotra | karan@gmail.com  | BCA              |        2
         8 | Sneha Jain     | sneha@gmail.com  | MCA              |        1
         1 | Rahul Kumar    | rahul@gmail.com  | Computer Science |        1
(8 rows)
```

```sql
SELECT * FROM Faculty;
```

```
 facultyid |       name       |      email       |       department
-----------+------------------+------------------+-------------------------
       101 | Dr. Amit Sharma  | amit@gmail.com   | Computer Science
       102 | Dr. Pooja Verma  | pooja@gmail.com  | Mathematics
       103 | Dr. Rajesh Kumar | rajesh@gmail.com | Information Technology
       104 | Dr. Neelam Singh | neelam@gmail.com | Artificial Intelligence
(4 rows)
```

```sql
SELECT * FROM Course;
```

```
 courseid |         coursename          | credits | facultyid
----------+-----------------------------+---------+-----------
      201 | Data Structures             |       4 |       101
      202 | Database Management System  |       4 |       103
      203 | Python Programming          |       3 |       101
      204 | Mathematics                 |       3 |       102
      205 | Artificial Intelligence     |       4 |       104
(5 rows)
```

```sql
SELECT * FROM Exam;
```

```
 examid | studentid | courseid |  examdate  | marks
--------+-----------+----------+------------+-------
    301 |         1 |      201 | 2026-10-10 |    85
    302 |         1 |      202 | 2026-10-11 |    90
    303 |         2 |      201 | 2026-10-10 |    78
    304 |         2 |      203 | 2026-10-12 |    88
    305 |         3 |      202 | 2026-10-11 |    65
    306 |         3 |      204 | 2026-10-13 |    72
    307 |         4 |      201 | 2026-10-10 |    92
    308 |         4 |      205 | 2026-10-14 |    89
    309 |         5 |      203 | 2026-10-12 |    75
    310 |         6 |      204 | 2026-10-13 |    95
    311 |         6 |      205 | 2026-10-14 |    81
(11 rows)
```

```sql
SELECT 'Student' AS TableName, COUNT(*) AS Total FROM Student
UNION ALL
SELECT 'Faculty', COUNT(*) FROM Faculty
UNION ALL
SELECT 'Course', COUNT(*) FROM Course
UNION ALL
SELECT 'Exam', COUNT(*) FROM Exam;
```

```
 tablename | total
-----------+-------
 Student   |     8
 Faculty   |     4
 Course    |     5
 Exam      |    11
(4 rows)
```

```sql
SELECT Student.StudentID, Student.Name
FROM Student
LEFT JOIN Exam
ON Student.StudentID = Exam.StudentID
WHERE Exam.ExamID IS NULL;
```

```
 studentid |      name
-----------+----------------
         8 | Sneha Jain
         7 | Karan Malhotra
(2 rows)
```

3. Implementation of Where Clause-

(Uses Q1–Q2 data only — no new tables/data) Write and run: (a) all students from a specific Branch using WHERE, (b) all exam records sorted by Marks descending using ORDER BY, (c) average Marks per CourseID using GROUP BY and AVG, (d) the highest Marks scored in each Course using GROUP BY and MAX.

(a) Find all students from a specific Branch using WHERE-

```
SELECT *
FROM Student
WHERE Branch = 'MCA';
```

```
 studentid |     name     |      email       | branch | semester
-----------+--------------+------------------+--------+----------
         2 | Priya Sharma | priya@gmail.com  | MCA    |        1
         4 | Neha Singh   | neha@gmail.com   | MCA    |        2
         6 | Anjali Mehta | anjali@gmail.com | MCA    |        1
         8 | Sneha Jain   | sneha@gmail.com  | MCA    |        1
(4 rows)
```

(b) Display all Exam records sorted by Marks in descending order-

```
SELECT *
FROM Exam
ORDER BY Marks DESC;
```

```
 examid | studentid | courseid |  examdate  | marks
--------+-----------+----------+------------+-------
    310 |         6 |      204 | 2026-10-13 |    95
    307 |         4 |      201 | 2026-10-10 |    92
    302 |         1 |      202 | 2026-10-11 |    90
    308 |         4 |      205 | 2026-10-14 |    89
    304 |         2 |      203 | 2026-10-12 |    88
    301 |         1 |      201 | 2026-10-10 |    85
    311 |         6 |      205 | 2026-10-14 |    81
    303 |         2 |      201 | 2026-10-10 |    78
    309 |         5 |      203 | 2026-10-12 |    75
    306 |         3 |      204 | 2026-10-13 |    72
    305 |         3 |      202 | 2026-10-11 |    65
(11 rows)
```

(c) Calculate average Marks per CourseID using GROUP BY and AVG-

```sql
SELECT
    CourseID,
    AVG(Marks) AS Average_Marks
FROM Exam
GROUP BY CourseID
ORDER BY CourseID;
```

```
 courseid |    average_marks
----------+---------------------
      201 | 85.0000000000000000
      202 | 77.5000000000000000
      203 | 81.5000000000000000
      204 | 83.5000000000000000
      205 | 85.0000000000000000
(5 rows)
```

(d) Find the highest Marks scored in each Course using GROUP BY and MAX-

```sql
143
144  SELECT
145      CourseID,
146      MAX(Marks) AS Highest_Marks
147  FROM Exam
148  GROUP BY CourseID
149  ORDER BY CourseID;
150
151
```

```
 courseid | highest_marks
----------+---------------
      201 |            92
      202 |            90
      203 |            88
      204 |            95
      205 |            89
(5 rows)
```